GoPilot

Frontend für Moodle, das das selbstgesteuerte Lernen von Auszubildenden an berufsbildenden Schulen fördert. Inspiriert von scobees.

- Fachlicher Kontext und Glossar: CONTEXT.md
- Architekturentscheidungen: docs/adr/
- Umsetzungsplan: docs/umsetzungsplan-v1.md
- Anleitung für Lehrkräfte und Moodle-Administration: docs/anleitung/lehrkraefte-moodle.md
- Anleitung zur App für Auszubildende und Lehrkräfte: docs/anleitung/app-gopilot.md (auch als PDF)
- Installation auf einem Moodle-System: docs/installation.md (auch als PDF)

Aufbau

```
plugin/local_gopilot/   Moodle-Plugin (PHP): Daten, Webservices, Ampel, Push
app/                    PWA (React + TypeScript + Vite)
deploy/dev/             Lokale Moodle-Entwicklungsumgebung (moodle-docker)
```

Lokale Entwicklung

Voraussetzungen: Git, Docker Desktop, Node.js 22 oder neuer, eine Bash (unter Windows: Git Bash).

Moodle starten

```
deploy/dev/setup.sh
```

```
deploy/dev/start.sh
```

`setup.sh` holt einmalig moodle-docker und Moodle 4.5 LTS nach `.dev/`. `start.sh` startet die Container, installiert Moodle beim ersten Mal und legt Testdaten an:

- Kurse `GOPILOT-LF5` (zwei Lernsituationen) und `GOPILOT-LF8` (eine Lernsituation), mit Soll-Terminen rund um die aktuelle Woche
- Lehrkraft `lehrkraft1` und Azubis `azubi1` bis `azubi3` mit unterschiedlichen Lernständen

Die Zugangsdaten der Testkonten stehen in deploy/dev/seed.php. Moodle läuft danach auf http://localhost:8000.

Der Moodle-Code liegt im Docker-Volume `gopilot_moodlecode`, weil Dateizugriffe auf das Windows-Laufwerk Moodle stark ausbremsen. `start.sh` kopiert ihn beim ersten Start aus `.dev/moodle` hinein. Das Plugin ist dagegen direkt aus `plugin/local_gopilot` eingehängt. Nach einer Änderung an `version.php` übernimmt ein erneuter Aufruf von `start.sh` das Upgrade.

```
deploy/dev/stop.sh
```

`stop.sh --reset` löscht zusätzlich Datenbank und Moodle-Daten.

App starten

```
cd app && npm install && npm run dev
```

Die App läuft auf http://localhost:5173 und spricht Moodle über `VITE_MOODLE_URL` aus `app/.env.development` an. Moodle erlaubt den Zugriff von fremden Ursprüngen an `login/token.php` und `webservice/rest/server.php` selbst (ADR 0007). Anmelden kannst du dich mit den Testkonten aus `deploy/dev/seed.php`.

Ohne Moodle geht es mit Beispieldaten im Browser:

```
cd app && npm run dev:mock
```

Im Mock-Modus melden sich `ada`, `ben` und `can` (Lernende mit grüner, gelber und roter Ampel) sowie `lena` (Lehrkraft) mit beliebigem Kennwort an.

Push-Benachrichtigungen funktionieren nur in der gebauten App (`npm run build && npm run preview`), weil der Dev-Server keinen Service Worker aktiviert.

Schnittstelle

Plugin und App halten sich an den API-Vertrag in docs/api-v1.md. Änderungen an Webservices dort zuerst eintragen.

Betrieb

Die vollständige Schritt-für-Schritt-Anleitung steht in docs/installation.md (auch als PDF): Voraussetzungen, Plugin installieren, Moodle einrichten, App bauen und ausliefern (nginx/Apache), Prüfung, Fehlersuche, Update und Deinstallation. Kurzfassung:

- **Moodle:** ab Version 4.2. Plugin nach `local/gopilot` kopieren, Upgrade ausführen, dann die Einrichtung aus dem Administrationsteil der Anleitung vornehmen. Für Web-Push einmalig VAPID-Schlüssel erzeugen: `php local/gopilot/cli/generate_vapid.php`. Unter **Website-Administration** → **Plugins** → **Lokale Plugins** → **GoPilot** die Adresse der App (`appurl`) eintragen, damit Moodle-Nachrichten auf die App verlinken.
- **App:** `npm run build` erzeugt statische Dateien in `app/dist/`. `VITE_MOODLE_URL` beim Build auf die Moodle-Adresse setzen. Auf der App-Domain ist HTTPS nötig, und der Webserver muss unbekannte Pfade an `index.html` weitergeben (SPA-Fallback, bei nginx `try_files $uri /index.html;`), damit Links aus Benachrichtigungen wie `/woche/2026-11-02` funktionieren.

Tests

```
cd app && npm run lint && npm test && npm run build
```

Plugin-Tests (PHPUnit, Code-Checker) laufen in der CI über `moodle-plugin-ci`, siehe .github/workflows/ci.yml.